

Time Series Data



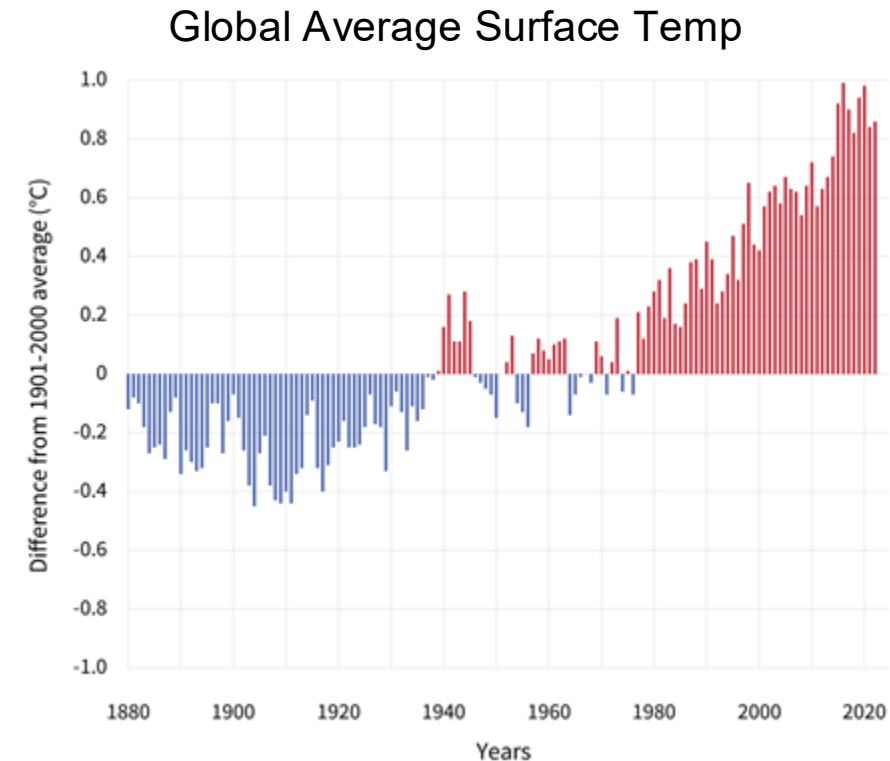
- What is Time Series Data?
- Processing Time Series Data
- Predicting Time Series Data
 - STL Decomposition
 - Stationarity and Autocorrelation
 - ARMA/ARIMA Models

What is Time Series Data?

Time Series Data



- **Time series data**: Data points collected over a range of time
 - Data may be collected at regular or irregular intervals
 - e.g. stock price, audio, natural language, signal data
 - Key property: Temporal ordering
 - Time series data change over time
 - Chronological order matters (past -> present -> future)
- How to process a feature that is time series data?
- How to predict time series data?



Processing Time Series Data

Recap: Feature Engineering



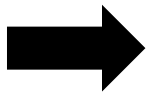
- **Feature Engineering**: The process of selecting, transforming, or creating input variables to improve the performance of a machine learning model

Transforming
features

- Transform the data for the model to handle inputs easier
 - Scaling, log-transformation
 - Encoding

Creating
features

- Make new features to improve model's performance
 - Cross/combine existing features
 - Use domain knowledge to create new features



How to handle time series data as a feature?

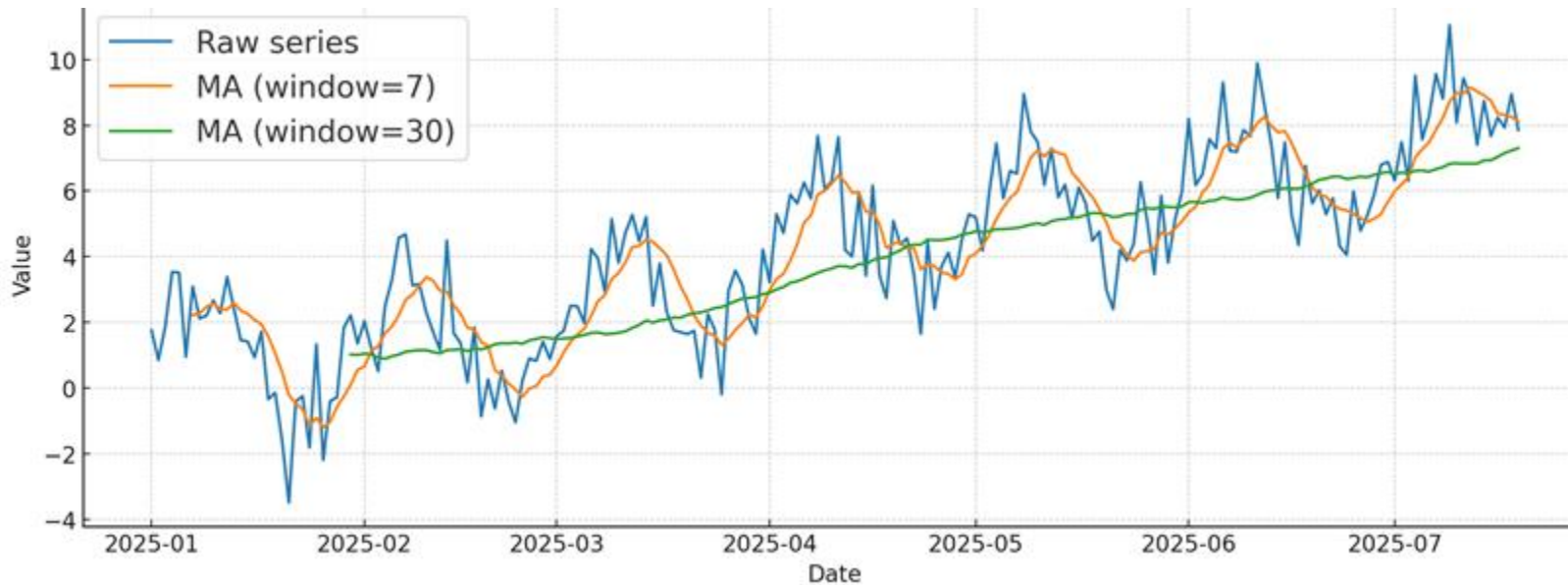
To notebook

Moving Average



A smoothing method by replacing each data point with a local average.

- **Moving average:** a local mean over certain timesteps (window)
 - In contrast to a simple mean, it can reduce noise and reveal local trend
 - Larger window: more smoothing, larger lag (slower to follow shift in data)
 - Smaller window: less smoothing, more noise



Cyclical Encoding



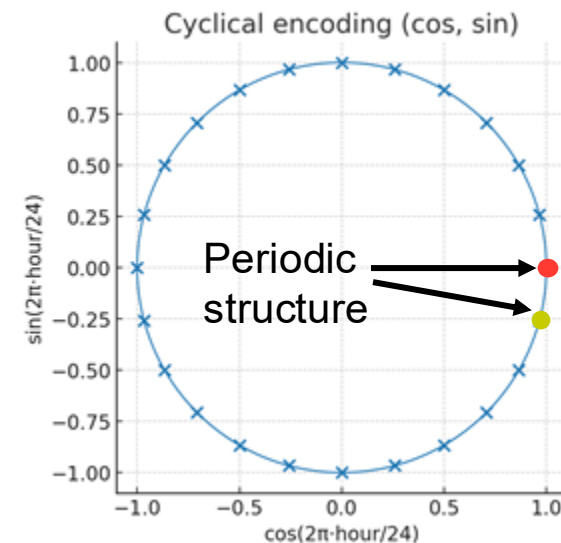
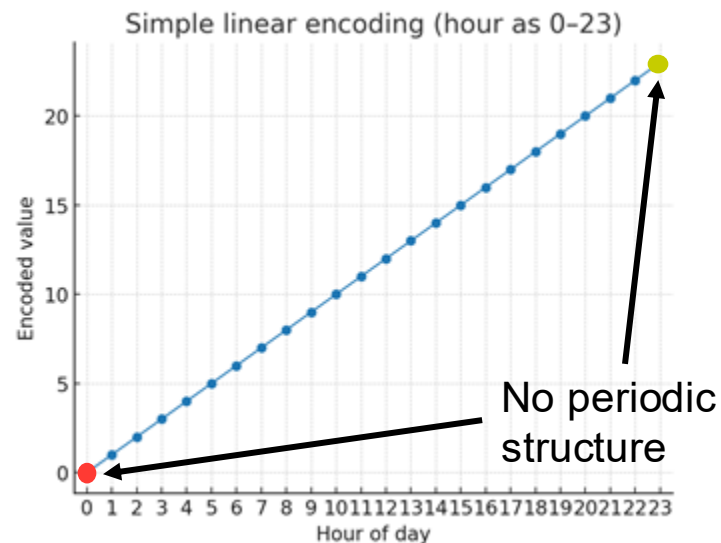
Encoding method using trigonometric functions to better capture periodic structure.

- Cyclical encoding captures periodic structure better than raw values
 - e.g. 23:00PM and 0:00AM=24:00PM is close
- Formally,

$$x_{sin} = \sin\left(\frac{2\pi t}{P}\right), x_{cos} = \cos\left(\frac{2\pi t}{P}\right)$$

where t is the value, P is the period

- e.g. hour: $P = 24$, week: $P = 7$, etc.



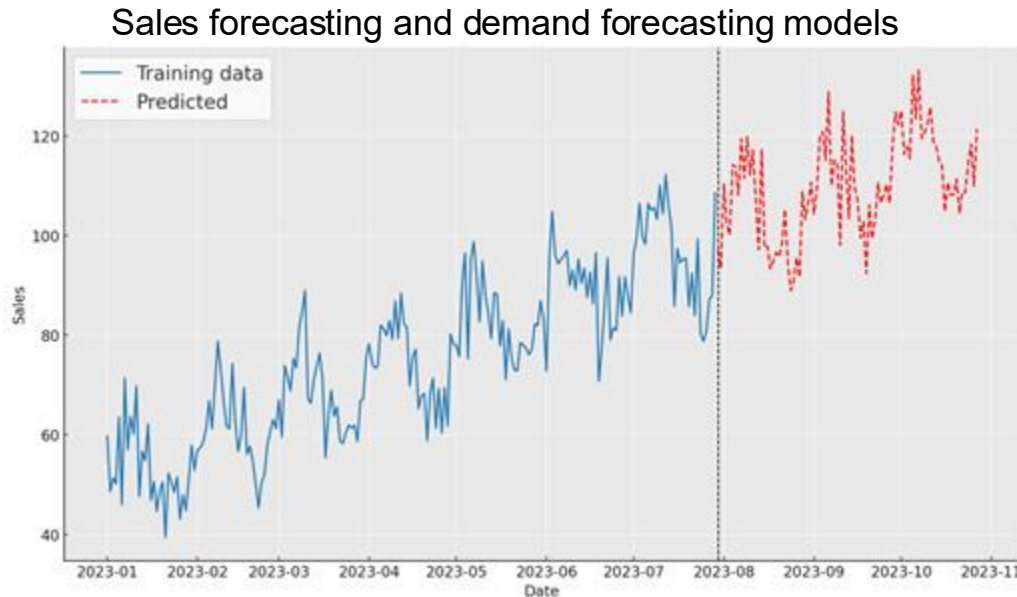
To notebook

- Moving average is useful for quick trend estimation, de-noise before modeling, create rolling features
 - Larger window = stronger smoothing but larger lag
- Cyclical encoding is useful to capture periodical structure of data
 - Encode using sine and cosine functions
 - Choose period P to match true cycle (24, 7, 365, etc.)

Predicting Time Series Data

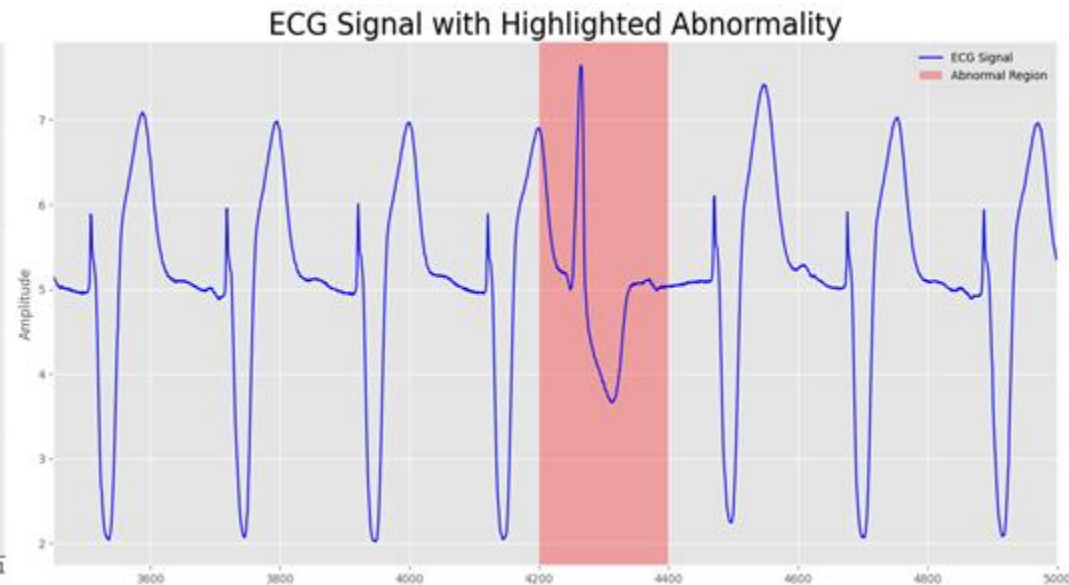
Motivation for Time Series Analysis

Analyzing periodical/seasonal patterns from data and modeling for forecasting and anomaly detection helps in early problem detection and planning.



Future Prediction

- Formulate business plans
- Optimize inventory management
- etc.



Anomaly detection

- Early detection of factory equipment
- Abnormal changes in power usage
- etc.

Statistical Models

- ✓ Assume certain statistical properties
- ✓ Excel at interpretability
- ✗ Can fail to capture complex nonlinear patterns

Examples

- ARIMA models:
combines autoregression, differencing, and moving average
- State-space models:
explicitly model latent variables that drive observed data

Machine Learning Models

- ✓ No assumption of statistical properties
- ✓ Can learn complex nonlinear patterns
- ✗ Often require large amounts of data
- ✗ Hard to interpret

Examples

- Non-time series methods:
e.g. Random Forests
- Machine learning models:
deep learning models (e.g. RNN, LSTM, Transformer)

Statistical Models

- ✓ Assume certain statistical properties
- ✓ Excel at interpretability
- ✗ Can fail to capture complex nonlinear patterns

Examples

- ARIMA models:
combines autoregression, differencing, and moving average
- State-space models:
explicitly model latent variables that drive observed data

Focus of this lecture

Machine Learning Models

- ✓ No assumption of statistical properties
- ✓ Can learn complex nonlinear patterns
- ✗ Often require large amounts of data
- ✗ Hard to interpret

Examples

- Non-time series methods:
e.g. Random Forests
- Machine learning models:
deep learning models (e.g. RNN, LSTM, Transformer)

To notebook

Dependency on Lag: Autocorrelation

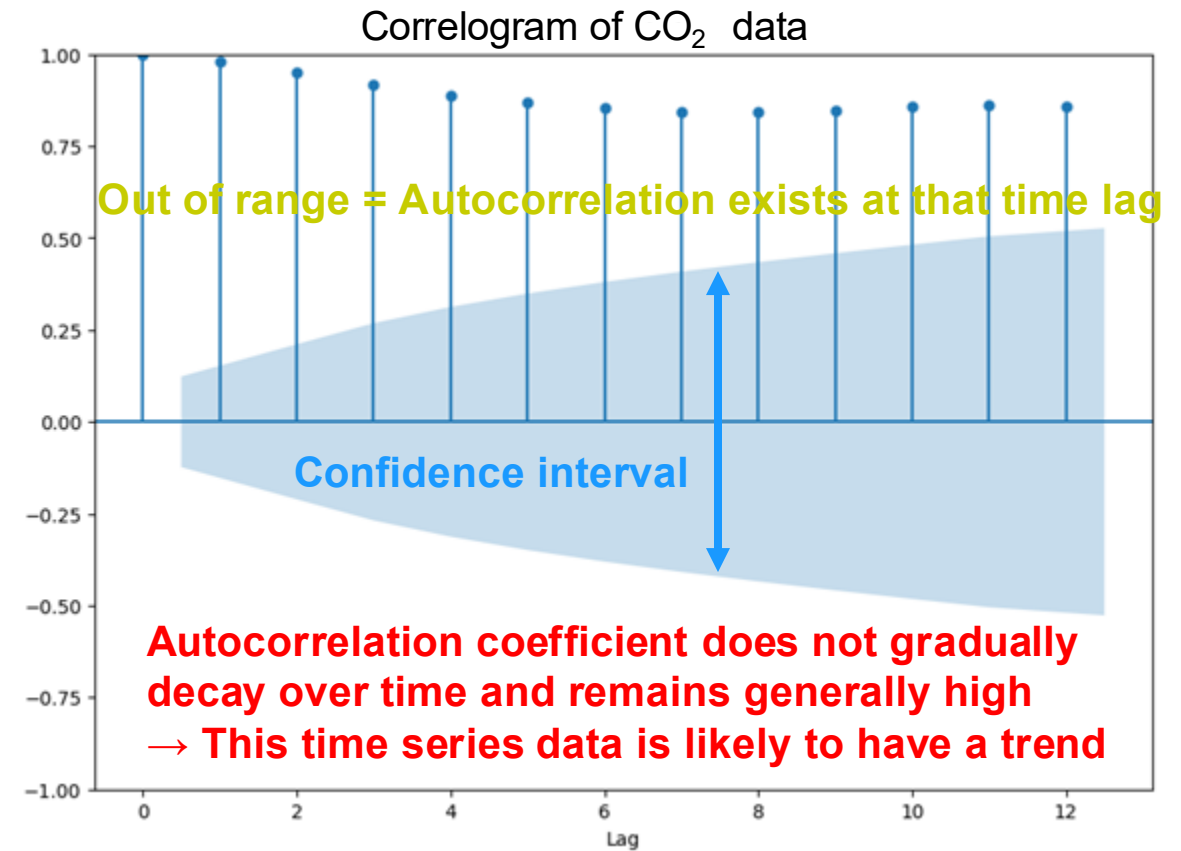
Autocorrelation coefficient: Calculates the correlation between the current data and the data from k periods ago. Trends or seasonality in the data are reflected in the autocorrelation.

$$\rho_k = \frac{\sum_{t=k+1}^n (X_t - \bar{X})(X_{t-k} - \bar{X})}{\sum_{t=1}^n (X_t - \bar{X})^2}$$

Calculating the autocorrelation coefficient reveals how much past data influences current data.

- Range: -1 to 1
 - Value is closer to 1 = Positive correlation
 - Value is closer to -1 = Negative correlation
 - Value is around 0 = No correlation

Correlogram plots the autocorrelation coefficients with lag as the x-axis.

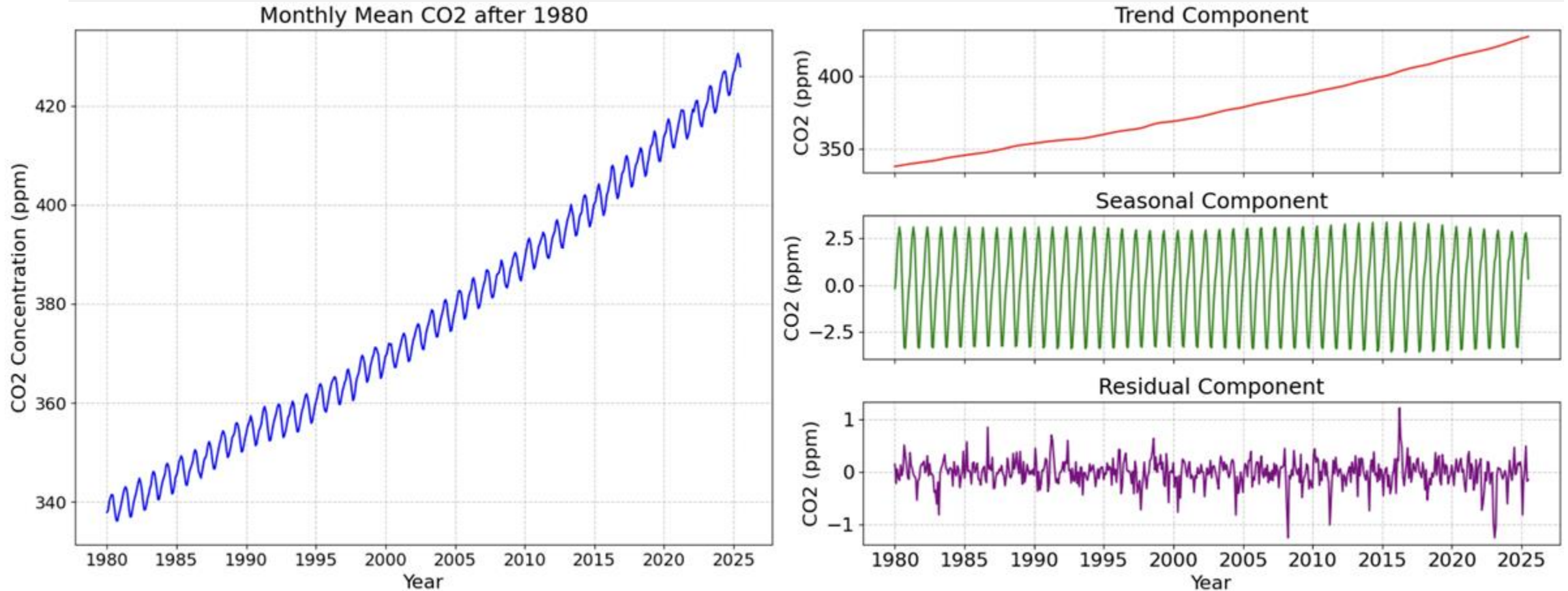


To notebook

STL Decomposition

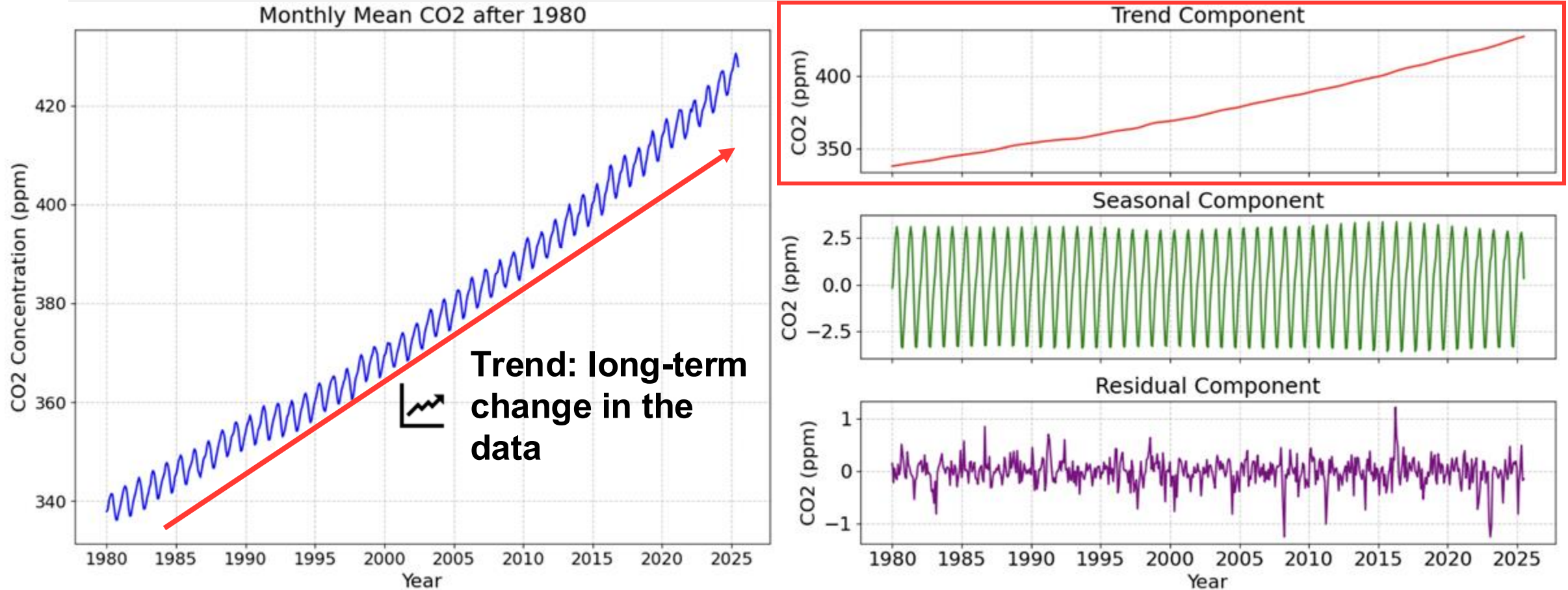


STL decomposition splits a series into **trend**, **seasonal**, and **residual** components, a useful method when seasonality or trend evolve over time.



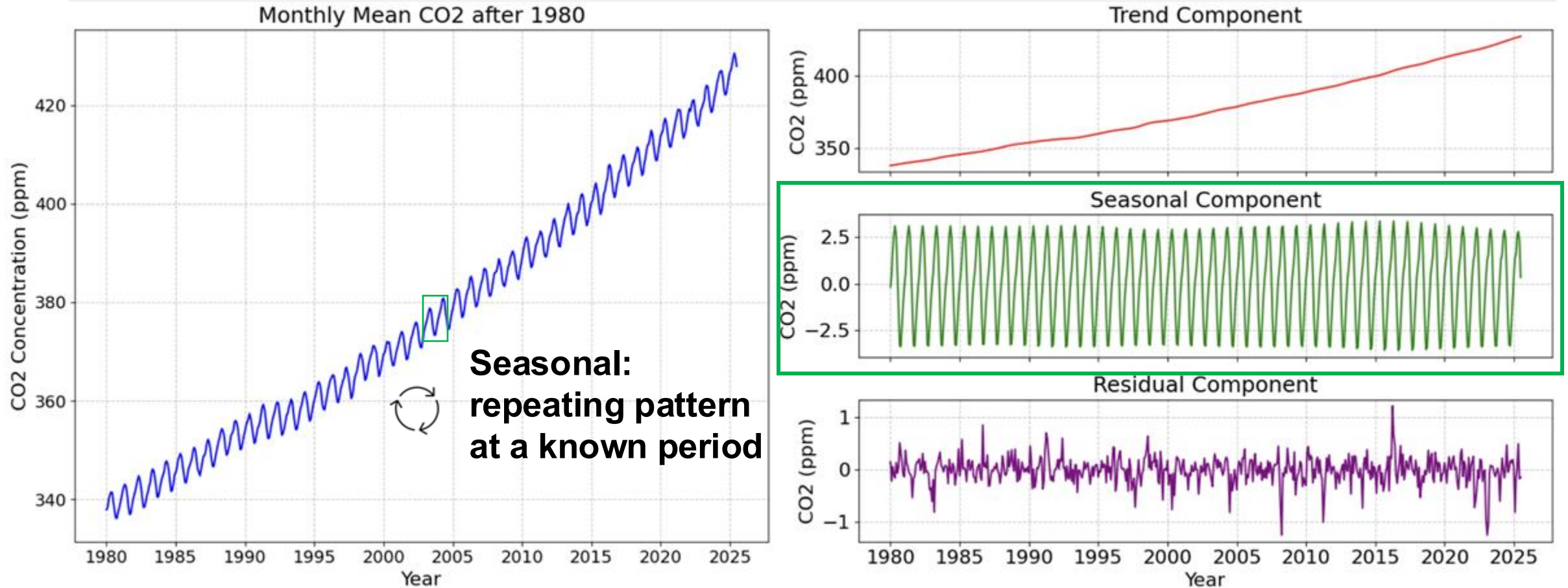
STL Decomposition

STL decomposition splits a series into **trend**, **seasonal**, and **residual** components, a useful method when seasonality or trend evolve over time.



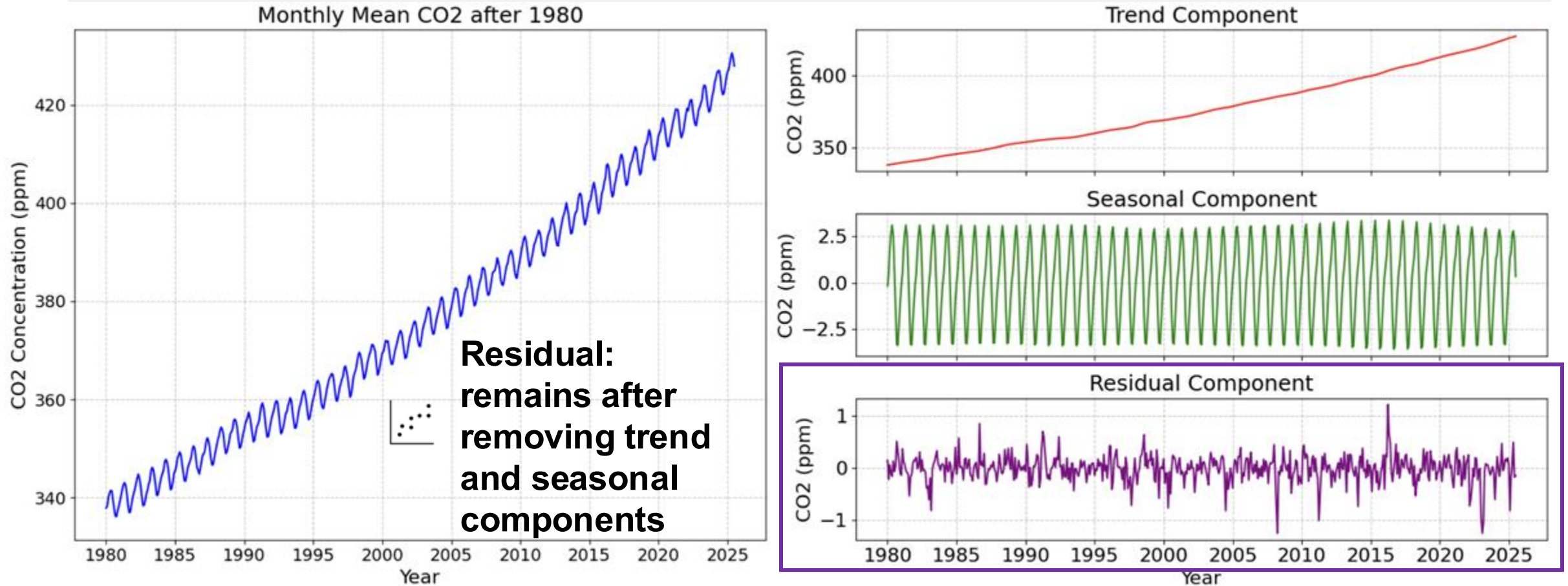
STL Decomposition

STL decomposition splits a series into **trend**, **seasonal**, and **residual** components, a useful method when seasonality or trend evolve over time.



STL Decomposition

STL decomposition splits a series into **trend**, **seasonal**, and **residual** components, a useful method when seasonality or trend evolve over time.

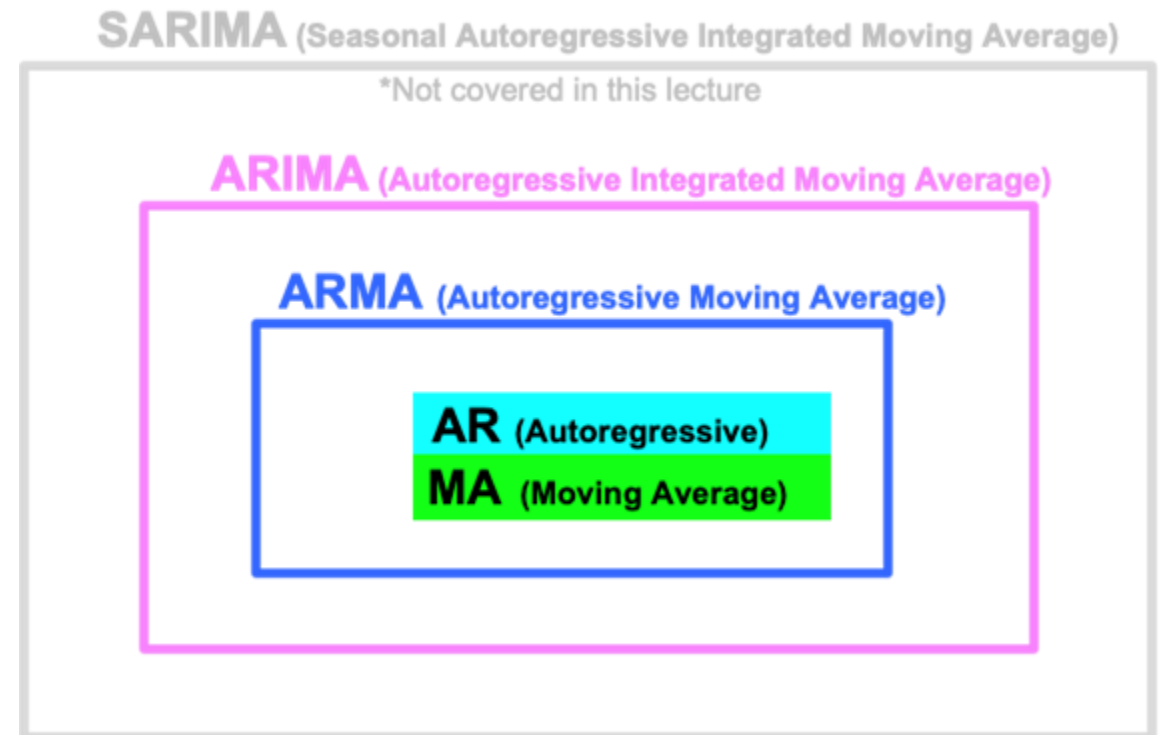


To notebook

Building a statistical model is a crucial part in making time series prediction. The introduce four models: AR, MA, ARMA, and ARIMA.

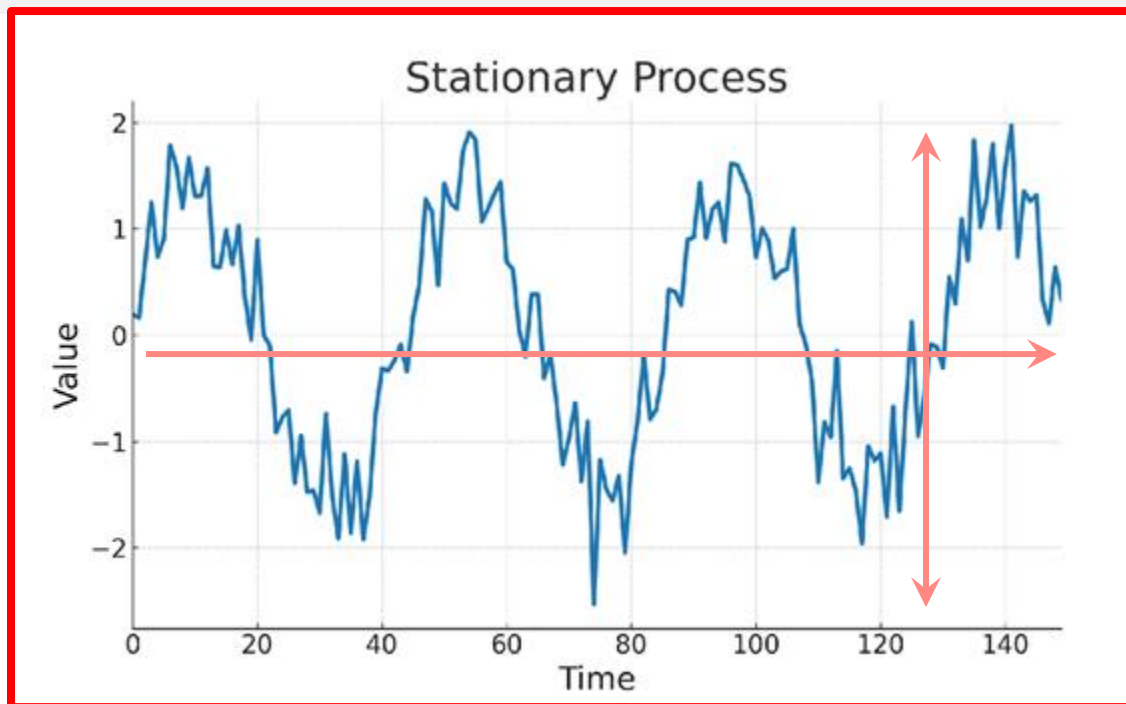
■ Classical time series models

- AR (autoregressive) model
- MA (moving average) model
- ARMA (=AR+MA) model
- ARIMA (=AR + I + MA) model

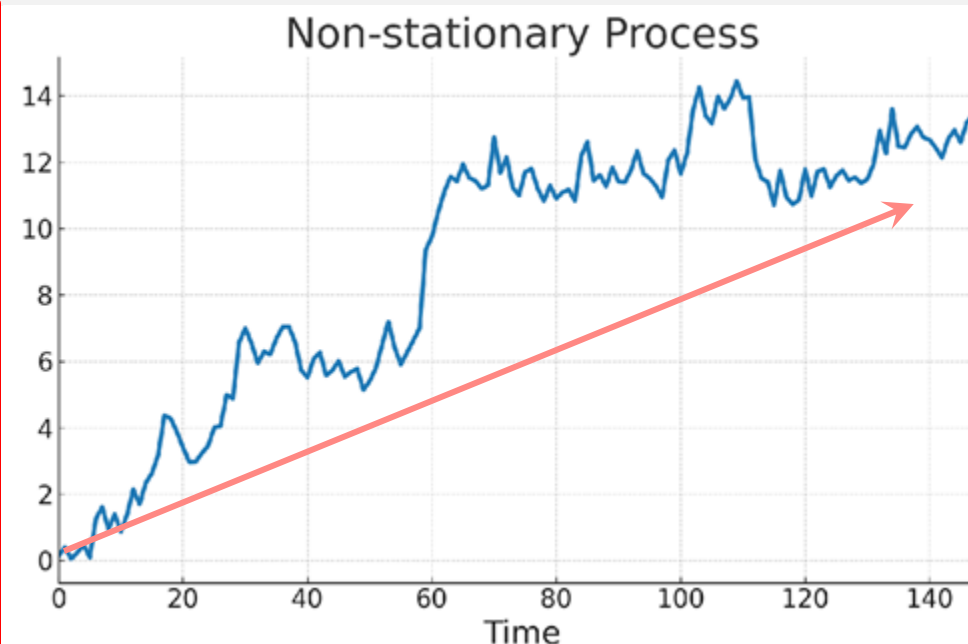


Stationary vs Non-stationary

Before diving into statistical modeling, we go over two types of time series: stationary vs non-stationary. Most statistical models assume stationarity.



- Statistical properties stay roughly constant

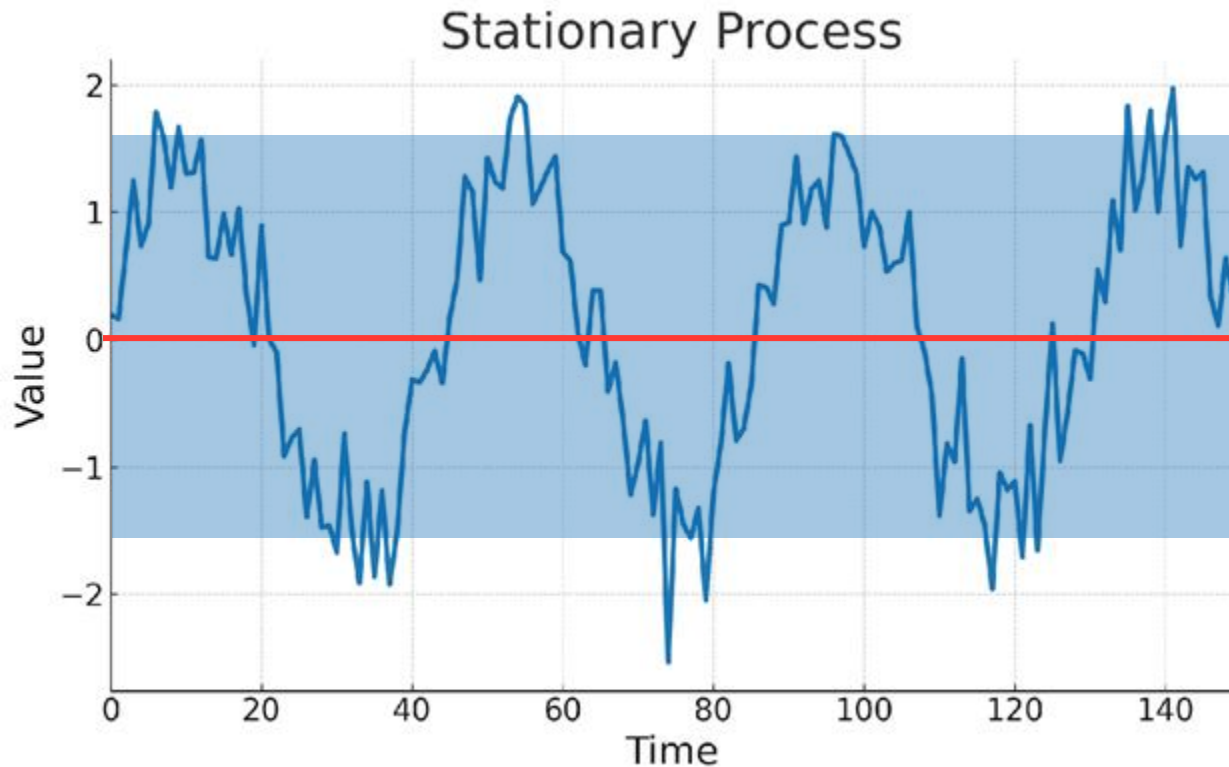


- Statistical properties change over time

➡ Most statistical models assume the time series is stationary!!

Stationary time series

Stationarity = statistics (mean, variance, autocovariance) stay constant over time.



■ Constant mean

Data fluctuates around a fixed central value

- no change in mean
- no persistent upward/downward trend

■ Constant variance

Spread of data remains roughly same

■ Constant autocovariance ($\hat{=}$ autocorrelation)

Correlations at a given lag are roughly stable

- How to check your time series is stationary?
→ Visual inspection and statistical tests (e.g., ADF test)

How to Check Stationarity: ADF Test

ADF (Augmented Dickey-Fuller) test examines whether a time series has a **unit root** (strong dependence on past values).

Equation

$$\Delta y_t = \alpha + \beta t + \gamma y_{t-1} + \delta_1 \Delta y_{t-1} + \cdot \cdot \cdot + \delta_n \Delta y_{t-n} + \varepsilon_t$$

- If $\gamma = 0$: there is a unit root $\rightarrow$ non-stationary
- If $\gamma < 0$: no unit root $\rightarrow$ stationary

How to interpret test results

- $p \geq 0.05$: cannot reject null $\rightarrow$ treat as **non-stationary**
- $p < 0.05$: reject null (unit root) $\rightarrow$ evidence for **stationarity** $\leftarrow$ **Null hypothesis: there is a unit root**

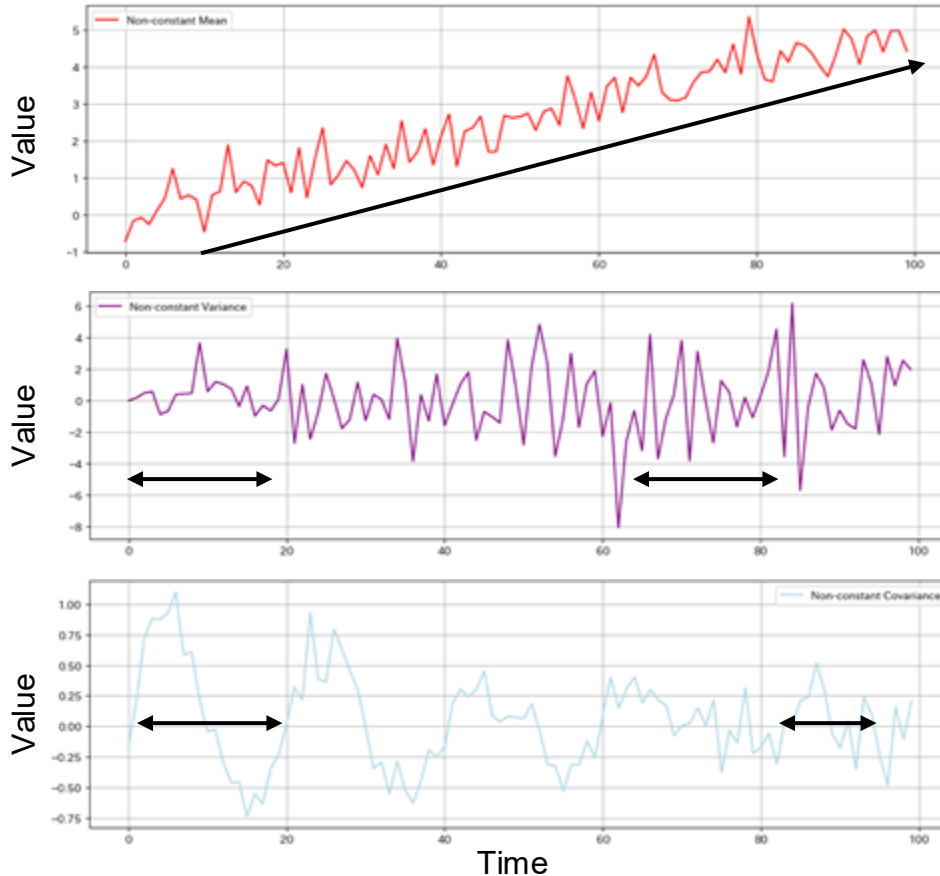
*Combine test result + visual judgement; don't rely on p-value alone.

Non-stationary Time Series



Non-stationarity = statistics (mean, variance, autocovariance) changes over time.

Examples with Non-stationarity



■ Inconstant mean

Data **does not** fluctuate around a fixed central value

- Change in mean
- e.g. **Persistent upward** trend

■ Inconstant variance

Spread of data **does not** remain roughly same

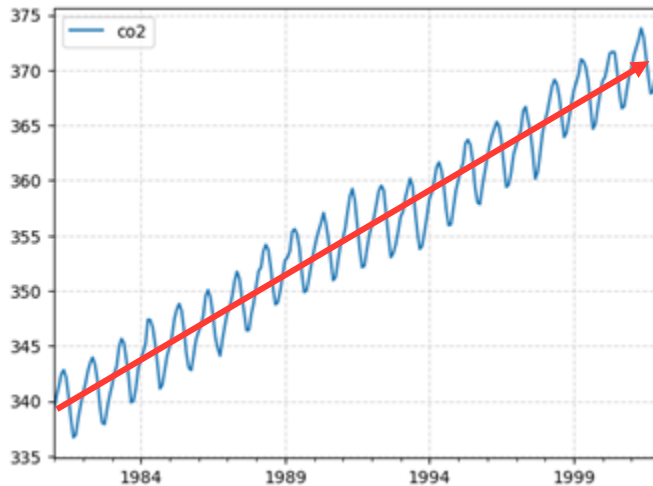
■ Inconstant autocovariance

Correlations at a given lag are **not** roughly stable

Transform Non-stationary Data into Stationary Data

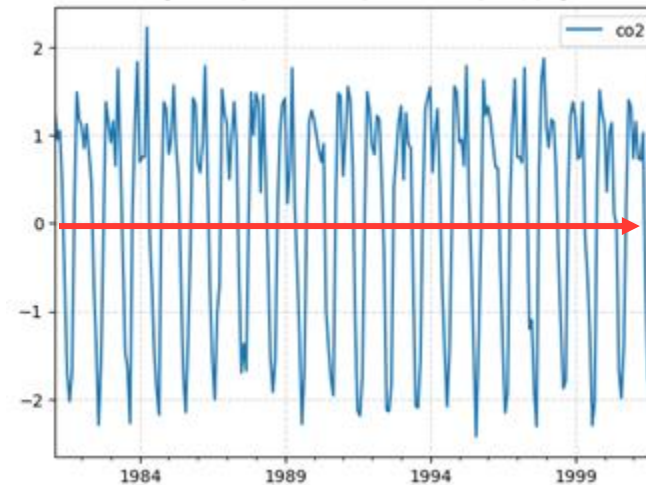
Apply **differencing** can remove trend in the data to make it stationary.

Original Data



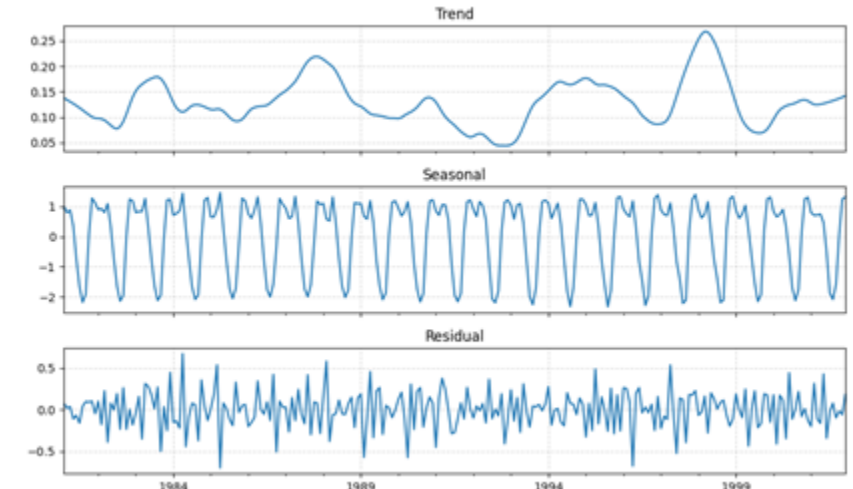
Shows upward trend.

Transformed Data ($\Delta X_t = X_t - X_{t-n}$)



**Removes trend
→ Data now fluctuates
around a stable level**

STL Decomposition

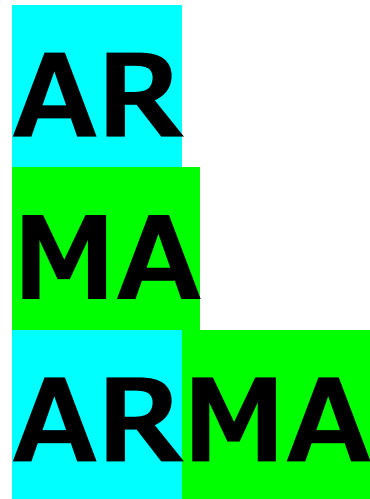


**Trend component is flat
→ Data is more stationary**

- If trend is more complex, try applying second differencing (Δ^2)

AR, MA, and ARMA models

AR, MA, and ARMA models are the basic models for **stationary** time series.
ARMA model is a combination of autoregression (AR) and moving average (MA).



SARIMA (Seasonal Autoregressive Integrated Moving Average)

*Not covered in this lecture

ARIMA (Autoregressive Integrated Moving Average)

ARMA (Autoregressive Moving Average)

AR (Autoregressive)

MA (Moving Average)

AR and MA models



AR and MA models are two most simple statistical models. They model the past influence in a different way and are the bedrocks that many statistical models are grounded on.

AR(p)

MA(q)

Parameter

p: # of lags to refer

q: # of past forecast errors
(residuals) to reference

Equation

$$y_t = c + \phi_1 y_{t-1} + \varepsilon_t \quad (p=1)$$

$$y_t = c + \varepsilon_t + \theta_1 \varepsilon_{t-1} \quad (q=1)$$

Meaning

The past explaining the present

Recent unpredictable shocks last

c : constant, ϕ, θ : coefficients, ε_t : noise at time t

ARMA (= AR+MA) model is the very standard model for stationary time series. Two parameters need to be specified.

$$\text{ARMA}(p, q) = \text{AR}(p) + \text{MA}(q)$$

e.g. ARMA (1,1)

$$y_t = c + \phi_1 y_{t-1} + \varepsilon_t + \theta_1 \varepsilon_{t-1}$$

ϕ : Autoregressive coefficient

θ : Moving average coefficient

ARIMA model further takes differences to handle **non-stationary data!**

SARIMA (Seasonal Autoregressive Integrated Moving Average)

*Not covered in this lecture

ARIMA (Autoregressive Integrated Moving Average)

ARMA (Autoregressive Moving Average)

AR (Autoregressive)

MA (Moving Average)

ARIMA

ARIMA = ARMA + differencing (I for integrated). This model assumes that the differencing process follows an ARMA model.

ARIMA(**p**, **d**, **q**)

d: # of differencing to remove trend or non-stationarity

e.g. ARIMA (**1**, **1**, **1**)

First difference

$$y'_t = \phi_1 y'_{t-1} + \varepsilon_t + \theta_1 \varepsilon_{t-1}$$

y'_t : first difference ($y'_t = y_t - y_{t-1}$)

ϕ : Autoregressive coefficient

θ : Moving average coefficient

Akaike Information Criterion (AIC)



Akaike Information Criterion (AIC) is one metric for parameter selection. AIC evaluates the trade-off between model complexity and accuracy.

$$AIC = 2k - 2 \ln(L)$$

The number of model parameters
(model complexity)

Likelihood (how well the model
explains the data)

*See Visualizing Data with Matplotlib
Appendix for details.

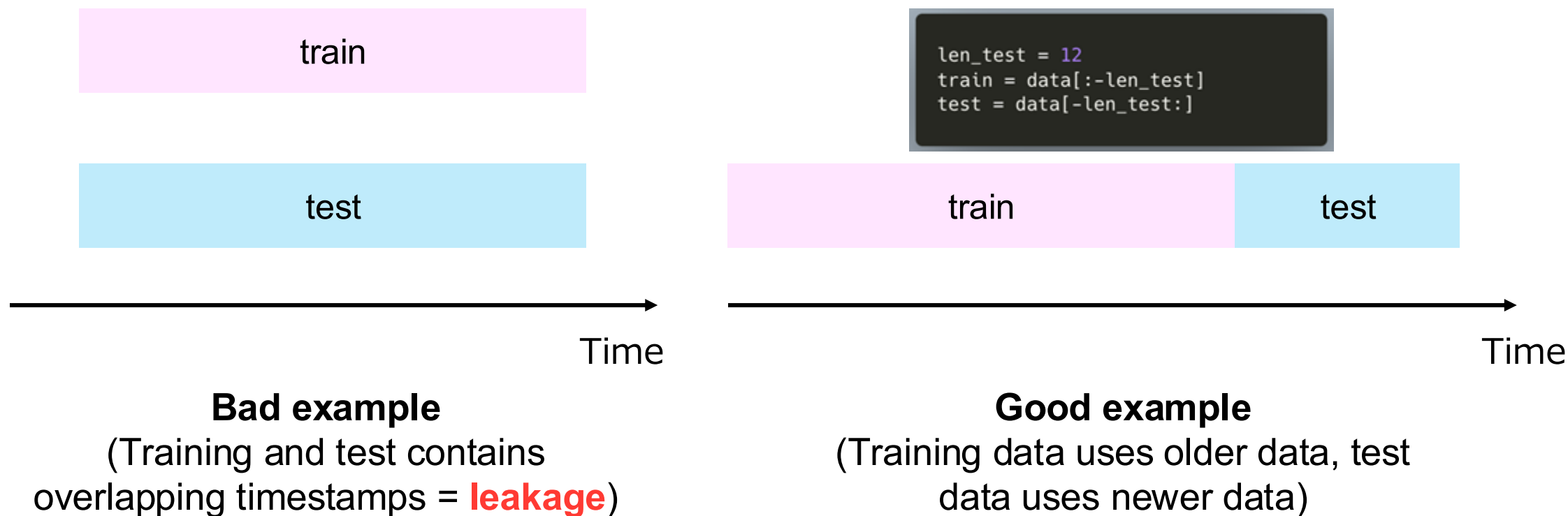
Model	p	d	q	k	-2ln(L)	AIC
ARIMA(1, 1, 1)	1	1	1	3	100	100+2×3=106
ARIMA(2, 1, 2)	2	1	2	5	90	90+2×5=100
ARIMA(3, 1, 3)	3	1	3	7	88	88+2×7=102

By minimizing AIC, a model with high accuracy can be selected while preventing overfitting.

Train/Test Split in Time Series Data



When using time series data (usually forecasting), make sure not to let future information leak into training (**data leakage**) — use time-aware splitting and validation.



*See p.48 for details.

To notebook

- Correlogram shows autocorrelation vs lag
- STL decomposition splits time series data into Seasonal, Trend, and Residual
 - Helps decide whether seasonal differencing is needed and deciding models to use
- Stationarity = stable mean, variance, and autocovariance over time
 - Test: visual checks + ADF test
 - Fix by differencing to remove trend
- ARIMA(p,d,q): **AR**(p) + **I**(d) differencing + **MA**(q)
 - Assumes difference is stationary
 - d: number of differencing to achieve stationarity
 - Selection: use information criteria (e.g. AIC) as guides
- If seasonality is complex or nonlinearities strong, consider other models (e.g. SARIMA, state-space models, or ML/deep alternatives).



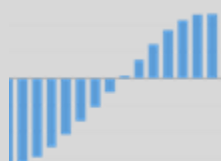
■ Compare predictive accuracy

- AIC measure model plausibility, not forecasting error → evaluate with holdout metrics (e.g. MSE)
- Benchmark with multiple-fitted models



■ Consider other models

- SARIMA: useful when clear seasonal structure exists (e.g. stocks)
- State-Space Models: good for latent dynamics, robust to noise
- ML / deep learning: capture complex data if you have enough data



■ Preprocessing & Explanatory work

- Inspect autocorrelation / partial autocorrelation
- FFT / periodogram for hidden periodicities
- Granger tests / cross-correlation analysis for causality analysis



Appendix

Appendix: Unit root



- Unit root: property that the influence of past data persists into the future
- **Unit root** makes a time series data **non-stationary**
- Example: AR model

$$y_t = \rho y_{t-1} + \varepsilon_t \longrightarrow \rho \text{ determines whether series is stationary or not}$$

- y_t : value at time t , ρ : coefficient term for previous value affecting current value, ε_t : error term (random term not predictable from the past)

Unit-root case: $\rho = 1$ (random walk)

||

$$y_t = y_{t-1} + \varepsilon_t$$

$$t = 1: y_1 = y_0 + \varepsilon_1$$

$$t = 2: y_2 = y_1 + \varepsilon_2 = (y_0 + \varepsilon_1) + \varepsilon_2$$

$$t = 3: y_3 = y_2 + \varepsilon_3 = (y_0 + \varepsilon_1 + \varepsilon_2) + \varepsilon_3$$

⋮

$$t = n: y_t = y_0 + \sum_{i=1}^t \varepsilon_i$$

→ Current value y_t is determined by sum of $\varepsilon_1 \sim \varepsilon_t$
= past random term have effect (non-stationarity)

Appendix: AR Model and ADF Test

ADF test checks whether past values have a permanent effect on future changes = whether a unit root (random-walk behavior) remains after removing trend and short-term dynamics

- Equation for ADF test

$$\Delta y_t = \alpha + \beta t + \gamma y_{t-1} + \delta_1 \Delta y_{t-1} + \dots + \delta_n \Delta y_{t-n} + \varepsilon_t$$

How this is derived:

- Step 1: Write AR model in difference form

$$y_t = \rho y_{t-1} + \varepsilon_t$$

$$y_t - y_{t-1} = \rho y_{t-1} + \varepsilon_t - y_{t-1} \quad \dots \text{Subtract } y_{t-1} \text{ from both sides}$$

$$\Delta y_t = \gamma y_{t-1} + \varepsilon_t \quad \dots \text{Set } \gamma = \rho - 1$$

- Step 2: Expand to ADF form

Add:

α : term to allow non-zero mean

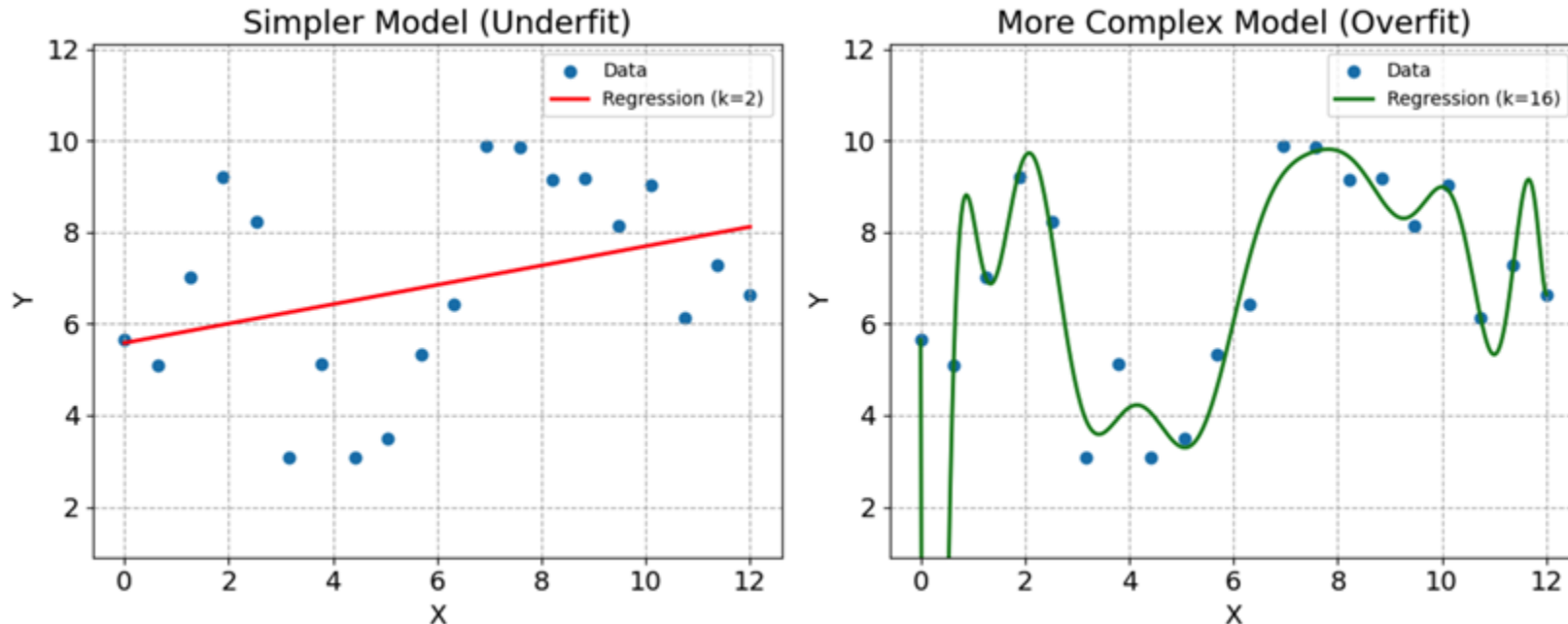
βt : allow deterministic trend

$\delta_i \Delta y_{t-i}$: lagged difference terms to absorb higher-order autocorrelation

Appendix: Why use AIC?

An overfitted model can achieve better fitness but is not necessary a model with good “predictive” ability.

- Example: increasing model complexity improves in-sample fit but risks fitting to noise



We want a metric to decide the hyperparameters

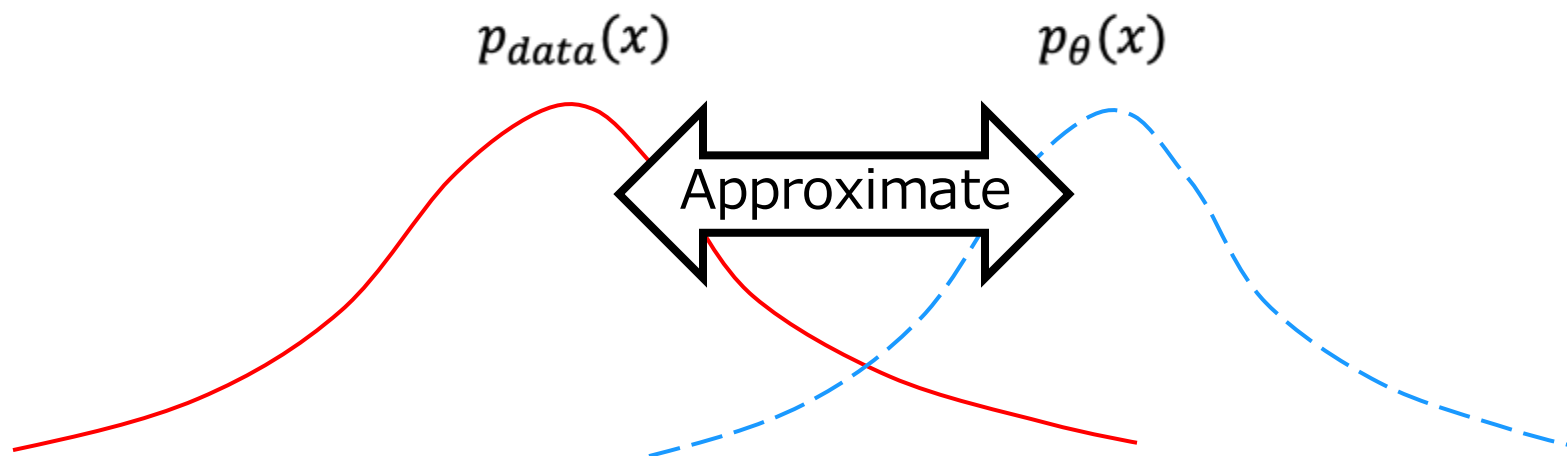


Use AIC

*Other options: cross validation ,etc.

Appendix: Deriving AIC (1/2)

Statistical modeling tries to make the model distribution $p_{\theta}(x)$ close to the true data distribution $p_{data}(x)$.



- One way to measure closeness of two distributions is **KL divergence**:

$$\begin{aligned} D_{KL}(p_{data}||p_{\theta}) &= \int p_{data}(x) \log \frac{p_{data}(x)}{p_{\theta}(x)} dx \\ &= \mathbb{E}_{p_{data}(x)}[\log p_{data}(x)] - \mathbb{E}_{p_{data}(x)}[\log p_{\theta}(x)] \end{aligned}$$

- x : data, p : true distribution, q : model distribution, θ : model parameters
- Since first term does not depend on θ , minimize KL = **maximize second term**

Appendix: Deriving AIC (2/2)

- The true distribution $p_{data}(x)$ is unknown, so $\mathbb{E}_{p_{data}(x)}[\log p_{\theta}(x)]$ cannot be computed
- Instead, we use the sample distribution $\hat{p}_{data}^N(x)$ and maximize the sample (log-)likelihood:

$$\log L(\theta) = \sum_{i=1}^N \log p_{\theta}(x), \quad \hat{\theta} = \arg \max \log L(\theta)$$

- $\log L(\theta)$ is biased upwards as an estimator of the expected log-likelihood because θ is trained on sampled data
- With large enough N , the bias can be approximated as
$$\mathbb{E}[\text{expected log-likelihood}] \approx \mathbb{E}[\log L(\hat{\theta})] - k$$
 - k : # of free parameters
- In relation, **AIC** is defined as

$$\text{AIC} = -2(\log L(\hat{\theta}) - k) = -2 \log L(\hat{\theta}) + 2k$$

Appendix: Interpreting AIC



Equation of AIC shows trade-off between (1) model fitness, and (2) model complexity.

- $$\text{AIC} = \underbrace{-2 \log L(\hat{\theta})}_{(1) \text{ Model fitness}} + \underbrace{2k}_{(2) \text{ Model complexity}}$$

- If fitness is high, (1) is high = AIC decreases
- If complexity is high, (2) is high = AIC increases
 - Acts as a penalty to prevent overfitting



- Model with minimum AIC = good fit × model not too complex
→ **Can be expected to generalize well to unseen data**

*AIC is a relative measurement, it does not ensure that model performance itself

Appendix: Implementing Train/Test Split for Time Series



For time-series forecasting never randomly split data. **Use time-aware splits** so training only uses past data and tests use future data.

- One way of splitting time series data is using scikit-learn's **TimeSeriesSplit**.

```
from sklearn.model_selection import TimeSeriesSplit

tscv = TimeSeriesSplit(n_splits=5)

for train_idx, test_idx in tscv.split(X):
    X_train, y_train = X[train_idx], y[train_idx]
    X_test = X[test_idx]

    model.fit(X_train, y_train)
    preds = model.predict(X_test)
```

